TDT4173 - Machine Learning

Project report

Yannick Rudolf (151732), Julien Gayraud (152237), Théo Choné (152286)
[Group 50] Bretzel Baguette

November 2025

Contents

1 Introduction **3**

2 Exploratory Data Analysis **4**
2.1 Domain knowledge 4
2.2 Check if the data is intuitive 5
2.3 Data generation 5
2.4 Exploration of individual features 6
2.5 Exploration of pairs and groups of features 7
2.6 Data cleaning 7

3 Feature Engineering **9**

4 Model Development **11**
4.1 LightGBM 11
4.2 XGBoost 12

5 Model Interpretation **14**

6 Conclusion **17**

1 Introduction

This machine learning project was conducted for Hydro, a Norwegian industrial company, in collaboration with Append Consulting. The aim is to get accurate forecasts of incoming raw material deliveries, which will be integrated into an optimization tool supporting Hydro's smelting operations.

The provided dataset contains records of receivals and purchase orders for various raw materials, each identified by a unique rm_id. The task is to predict the cumulative weight of deliveries for each material from January 1 to May 31, 2025, for any chosen end date within this period. Model performance is evaluated using the Quantile Error at level 0.8, an asymmetric loss function that penalizes overestimation more than underestimation, aligning with Hydro's preference for conservative forecasts.

The report presents the exploratory data analysis, feature engineering, model development, and model interpretation of the 2 chosen models, LightGBM and XGBoost. It concludes with a discussion of the results and potential improvements.

2 Exploratory Data Analysis

2.1 Domain knowledge

The dataset consists of historical records of raw material deliveries to Hydro's smelting facilities, spanning from 2004 to 2024. Each material rm_id is represented by a daily time series of received quantities net_weight. Deliveries are driven by production demand, supplier logistics, and operational schedules.

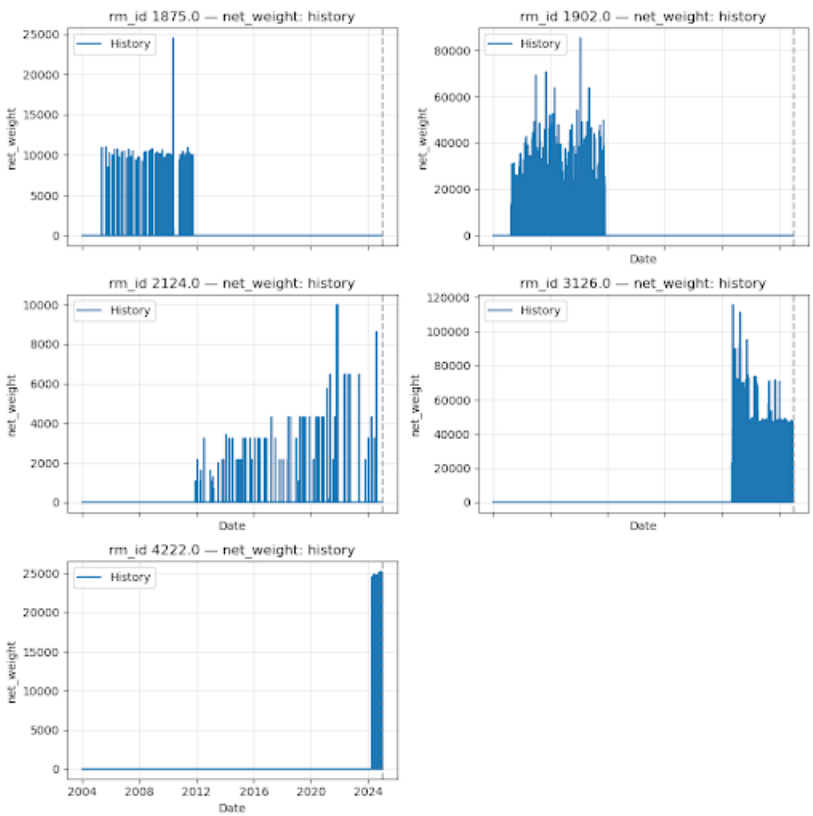

Figure 1: Historical daily net weight Trends for Selected RM IDs

We have several kinds of deliveries for different rm_id. Some are punctual and some are regular. Some started a long time ago and some did not. Some ended a long time ago and some continue to be done. We suggest adding features to catch these different behaviors : the delay between the 1st January 2025 and and the first/last delivery, the number of deliveries all time, the number of deliveries during the last year.

2.2 Check if the data is intuitive

Preliminary inspection ensured that the data was consistent and plausible.

All records with missing or non-positive delivery weights were excluded, and timestamps were converted to a daily format without timezone inconsistencies. Days without recorded deliveries were filled with zeros, ensuring temporal continuity. Only a few rows were useless and didn't make sense : missing net_weigth values, important id's (such as rm_id in materials.csv), and some negative values for deliveries or orders. Delivered quantities are consistent for a company (thousands of kg). We succeed to make most of the links between the dataframes. It means that for most of the orders there is a row in transportation.csv to describe the transportation. It is the same for materials : most of the rm_id are described in materials.csv. We succeed to merge files with the required ids.

This revealed that most materials displayed realistic and intermittent delivery patterns, with frequent zeros and occasional positive peaks.

2.3 Data generation

The project combined multiple data sources originating from Hydro's operational systems:

- Receivals data, containing the time series of daily delivered weights
- Material descriptions, detailing alloy family, treatment, and storage location
- Transportation records, providing mean transport weights and transporter identifiers
- Purchase orders, that we didn't managed to use because we didn't find a good correlation between orders and receivals using rm_id. Moreover, we didn't managed to link orders to the proper rm_ids because there are multiple product_id for one rm_id and vice versa. So we only used static features about orders.

The receivals data was first aggregated to daily granularity and then aligned on a uniform calendar shared by all materials. A filtering process removed materials that were inactive or too rarely delivered, defined as those with fewer than 30 recorded delivery days or with last activity before 2023. In total, around 96 materials were excluded for having insufficient history. This step ensured that only materials with meaningful temporal behavior were included in the modeling dataset.

2.4 Exploration of individual features

Exploration at the single-variable level focused on summary indicators computed per material rather than visual plots. For each *rm_id*, the total, mean, and median delivery weights, the total number of delivery days, and the time since last recorded delivery were calculated. These metrics revealed substantial variability between materials — some showed thousands of active days, while others had only sporadic deliveries. All computed statistics were within realistic operational ranges, and no outliers or anomalies were identified. This confirmed that the raw data was already consistent and interpretable at the individual feature level.

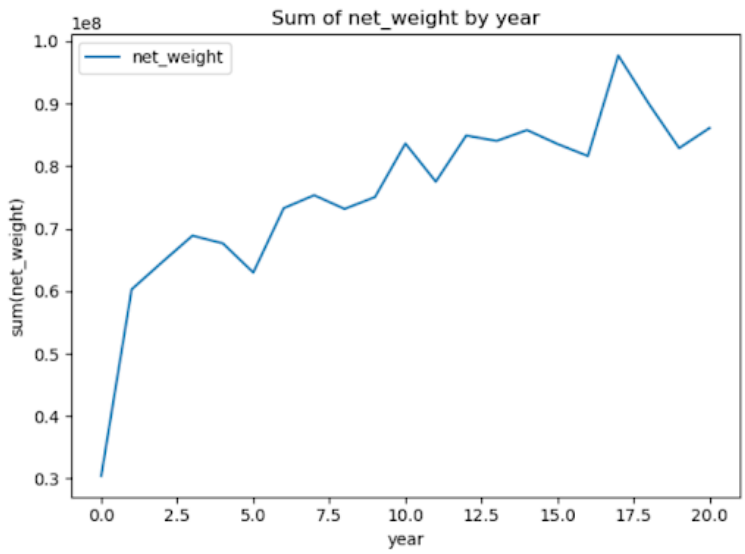

Figure 2: Total sum of net_weight through the years

There is a trend through the years. The amount of deliveries is increasing with the years. Feature added : year = current year - 2004

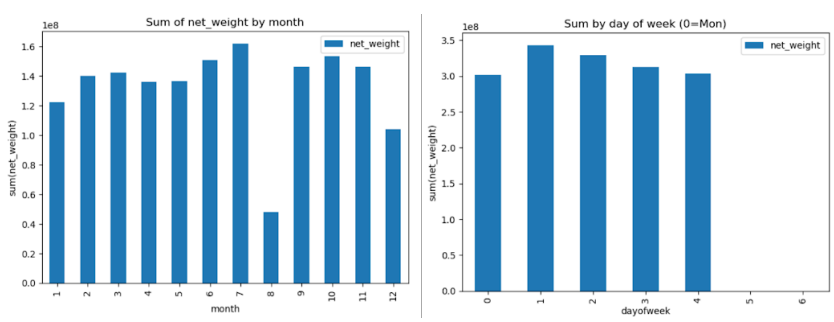

Figure 3: Total sum of net_weight for each month & for each day of the week

The month influences the deliveries. There are less deliveries in August and December. Features added : month, is_august, is_december. The day of the week influences the deliveries as well. We don't have any deliveries during the weekend. Features added : day_of_week, is_weekend

2.5 Exploration of pairs and groups of features

Temporal dependencies were investigated through a systematic lag correlation analysis. For each material, the correlation between daily deliveries and their lagged values was measured across hundreds of lag intervals. The analysis revealed strong autocorrelation at multiples of seven days, indicating a clear weekly delivery pattern across many materials. These findings confirmed that delivery behavior was periodic and guided the selection of predictive lag features used later in model training.

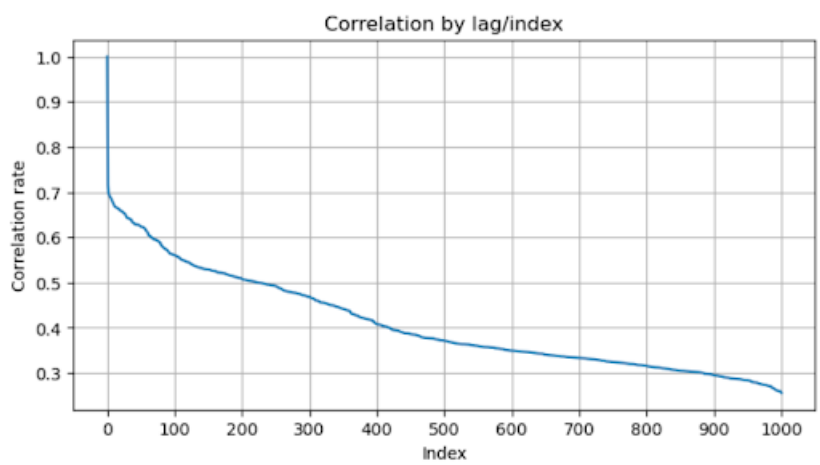

Figure 4: Ordered correlation rate between net_weight and its lags

The range of lags used is range(160,1000,7). The first boundary is 160 because we have to predict 155 days in 2025 and then with the lag 160 we still use the data in the history data set. The lag with the highest correlation rate is the lag 364. This lag is a multiple of 7 and it represents the delivery one year before. We arbitrarily chose all lags with a correlation rate greater than 0.5.

2.6 Data cleaning

A comprehensive preprocessing pipeline was applied to prepare the dataset for feature engineering and modeling. The main steps included:

- **Filtering and aggregation**: Removal of invalid values and aggregation of multiple deliveries on the same day

- **Calendar reindexing**: Alignment of all materials on a shared daily timeline from 2004 to 2024, with zeros inserted for non-delivery days
- **Rare material exclusion**: Filtering out materials with insufficient activity or outdated records
- **Feature enrichment**: Integration of static information (material and transport attributes)
- **Temporal feature generation**: Creation of lag-based and rolling window features (e.g., 7-, 14-, 30-, and 365-day windows) and calendar attributes (year, month, day of week, weekend indicator)
- **Sorting and optimization**: Ordering by date and material and converting numeric fields to optimized data types for efficiency
- **Remove unusable features**: String values (e.g. stock_location, raw_material_alloy), all ids, data_arrival, etc.

After cleaning, the resulting dataset consisted of continuous daily series for each active material, enriched with both temporal and static characteristics. This ensured a coherent and model-ready foundation for feature engineering and forecasting.

3 Feature Engineering

Based on the available data and the long-term forecast horizon, we explored several techniques to generate a strong set of features. The main challenge lies in developing features that are both correlated with the target and can still be applied to future dates. Therefore, the easiest features to include are date-based and static features.

The forecasting task introduces several unique constraints that significantly influence the feature engineering strategy:

- **Temporal Constraints**: Forecasts must cover the period from January 1, 2025, to May 31, 2025, using only information available up to December 31, 2024. This imposes a strict cutoff to prevent any data leakage from the future into the training features
- **Horizon Variability**: The task involves producing cumulative forecasts over varying time horizons (ranging from 1 to 150 days). As a result, the engineered features must remain robust and informative across multiple temporal scales.
- **Data Sparsity**: Many raw materials have irregular and infrequent delivery patterns, leading to extended gaps in the data. This irregularity reduces the effectiveness of conventional time series features and demands alternative feature construction methods.
- **Evaluation Asymmetry**: The quantile loss function penalizes overestimations more than underestimations. Consequently, the feature design must support more conservative prediction behavior to align with this asymmetric evaluation metric.

Given these constraints, we categorized the engineered features into three classes based on their applicability to future forecasting. **Class 1 (Fully Projectable Features)** includes date-based features (such as month or day of the week), static material characteristics, and lifetime historical aggregates. These can be computed entirely from existing information without making assumptions about future behavior and therefore form the foundation of both modeling pipelines. **Class 2 (Recursive Features)** consists of lag features and rolling window statistics, which depend on past observations but can be recursively updated during the forecast period using previously predicted values. Finally, **Class 3 (Semi-Projectable Features)** covers elements like purchase order information (representing known scheduled deliveries) and trend momentum indicators, which make partial use of known future inputs or extrapolate from historical trends to extend predictive insight.

Table 1: Generated features throughout the project

Feature	**Description**	**Class**
Temporal Features	Extract cyclical and calendar-based patterns from date structures. Include hierarchical time decomposition (year → quarter → month → week → day), cyclical indicators (day of week, month), and boundary markers (month/quarter start or end). Generated purely from forecast dates, requiring no historical data.	Class 1 – Fully Projectable
Lag Features	Create auto-regressive relationships using historical values at specific time offsets. Include seasonal (365 days), multi-year (728, 1092 days), and sub-annual (30–180 days) lags. Require recursive propagation beyond the historical boundary, introducing potential error accumulation.	Class 2 – Recursive
Rolling Window Aggregations	Compute summary statistics (mean, std, median, percentiles) over sliding temporal windows to capture trends, volatility, and stability. Window sizes (7–365 days) reflect short- to long-term business cycles.	Class 2 – Recursive
Frequency and Recency Metrics	Characterize delivery event timing through frequency counts, inter-event intervals, days since last delivery, and stability flags indicating regularity. Capture ordering consistency and replenishment cycles.	Class 2 – Recursive
Trend and Seasonality Features	Capture higher-level temporal dynamics such as growth rates, seasonal decompositions (e.g., Jan–May averages), and multi-horizon comparisons to detect acceleration or deceleration trends.	Class 3 – Semi-Projectable
Business Domain Features	Integrate operational and material-specific intelligence including purchase order data (scheduled deliveries, fulfillment rates), material properties (alloy, format), and logistics metadata (transportation, suppliers). Provide forward-looking signals.	Class 3 – Semi-Projectable
Static Aggregates	Compute long-term material-level summary statistics such as lifetime volume, average delivery size, historical volatility, and activity metrics (delivery counts, years of data). Establish stable behavioral baselines.	Class 1 – Fully Projectable

4 Model Development

At the start of the project, we explored many models to see which one performed the best. Our goal was to compare both classical time series models and machine learning regressors, and select the one that produced the most reliable and conservative forecasts according to the Quantile Error at 0.8 metric used on Kaggle. We experimented with the following predictor families:

- **Classical time series models**: ARIMA, Exponential Smoothing
- **Machine learning regressors**: Random Forest, CatBoost, LightGBM, XGBoost
- **Deep learning model**: Temporal Fusion Transformer (TFT)

We attempted to implement the TFT model, but due to implementation challenges and convergence issues, we were unable to obtain usable results. Consequently, our development focused on LightGBM and XGBoost, which produced stable and accurate forecasts.

4.1 LightGBM

In the notebook, we implemented LightGBM using the following workflow:

- **Feature Engineering**: We used the following features across multiple categories:
  - Historical features: Seasonal features including average Jan-May deliveries over 3 and 5 years, last year's Jan-May and total deliveries, and Jan-May growth rates. Trend features captured total weight and daily averages over 3-month, 6-month, and 12-month windows, along with trend comparisons between periods.
  - Variability features: Standard deviation, coefficient of variation (CV), median, and maximum delivery weights over 3-month periods to capture delivery stability patterns.
  - Purchase Order (PO) features: PO coverage rate, average delivery delays, order-to-receival time, PO fulfillment rates, closed order rates, active PO counts and quantities, expected delivery days, and 3-month PO statistics (11 PO-related features total).
  - Stability features: Binary indicator for regular delivery patterns and years active for each raw material.
  - Temporal features: Date-based features including month, quarter, day of week, day of month, day of year, week of year, and binary indicators for weekends, month/quarter starts and ends.
  - Recency and frequency features: Days since last delivery, delivery counts across different time windows (3mo, 6mo, 12mo), and average days between deliveries. In total, we used 47 features for model training.

- **Model Training**: Data was split temporally into training and validation sets, with September 1, 2024 as the split date. All data before this date was used for training (21,420 samples), while data from September 2024 onwards was used for validation (4,284 samples). For the final predictions on 2025 data, the model was retrained on the entire available historical dataset.

- **Hyperparameter Choices**: The objective function was set to 'quantile' with alpha=0.2 to directly optimize for the quantile loss metric used in evaluation. We used a learning rate of 0.02 with 31 leaves per tree. To prevent overfitting, we applied feature bagging (feature_fraction=0.8) and data bagging (bagging_fraction=0.8) with a bagging frequency of 5. The model used gradient boosting decision trees (GBDT) and was trained with early stopping (50 rounds patience) for up to 1,000 boosting rounds. The best model converged at iteration 340 with a validation quantile loss of 8,656.11.

- **Evaluation**: Predictions on the validation set were scored using Quantile Loss with q=0.2, along with MAE and RMSE for additional performance insights. Feature importance analysis revealed that long-term historical features were most influential: weight_12mo (12-month total weight) was by far the most important feature, followed by days_since_last_delivery, daily_avg_12mo, and deliveries_count_12mo. This highlighted the importance of long-term delivery patterns and recency information for accurate forecasting.

- **Forecasting**: We created a test dataframe containing all required features for each (raw material, forecast date, horizon) combination needed for 2025 predictions. Features were calculated using all historical data available up to each forecast date. The trained model generated predictions for each test sample, and predictions were constrained to non-negative values. The final predictions were formatted according to the submission requirements and saved to Kaggle for evaluation.

4.2 XGBoost

Similarly, XGBoost was applied as follows in the notebooks:

- **Feature engineering**: We used the most correlated lags within the last 1000 lags before (the current day - 160 days). We chose arbitrary lags with a correlation greater than 0.5 with the daily net_weight. It represents 128 lags. After a feature analysis, we added date features such as "day of the week", "day of the year", "year". We added some rolling mean and variance. We added features that represent the number of deliveries on a certain range (all time, last year, last month) and the delay between the 1st January 2025 and the last delivery in order to distinguish the rm_ids. Finally, we added dummies related to the transporter name, name

of the raw material, location of the raw material and family. There are 25 different transporters so we created dummies for these transporters. There are some float values that are not directly related to an rm_id such as "tare_weight" or "gross_weight". We chose to add some information about these column (mean) because it can hide information useful for an AI.

- **Model training**: Data was split temporally into training and validation sets. We trained the model on the data before 2024 and evaluated / tested the model on 2024. For the real evaluation (on 2025) we trained the model on the whole data set.

- **Hyperparameter choices**: The objective function is the "pseudohuber-ror" that is similar to the absolute error but it is a C1 function. This function is close to the teacher evaluation function (i.e. The quantile loss function with q=0.2) and gives accurate results. For each tree we chose to use 80% of the rows and 80% of the columns to not generate an over-fitted model that won't focus on only one column or rows with outliers (We used : subsample = 0.8 and colsample_bytree = 0.8). Hyperparameters such as tree depth, number of trees and learning rate were optimized using grid search. The best parameters found are n_estimators = 3000, learning_rate=0.02, max_depth = 8

- **Evaluation**: Predictions on the validation set were scored using Quantile Error with q=0.2 on the cumulative sum of predictions and RMSE . We also analyzed feature importance, which highlighted lag correlations and how much each feature is used.

- **Forecasting**: We created a new dataframe that contains required information to predict deliveries in 2025 and then predict the daily deliveries each day. After creating the cumulative sum we submitted the predictions to Kaggle for evaluation.

5 Model Interpretation

Feature analysis of the XGB model

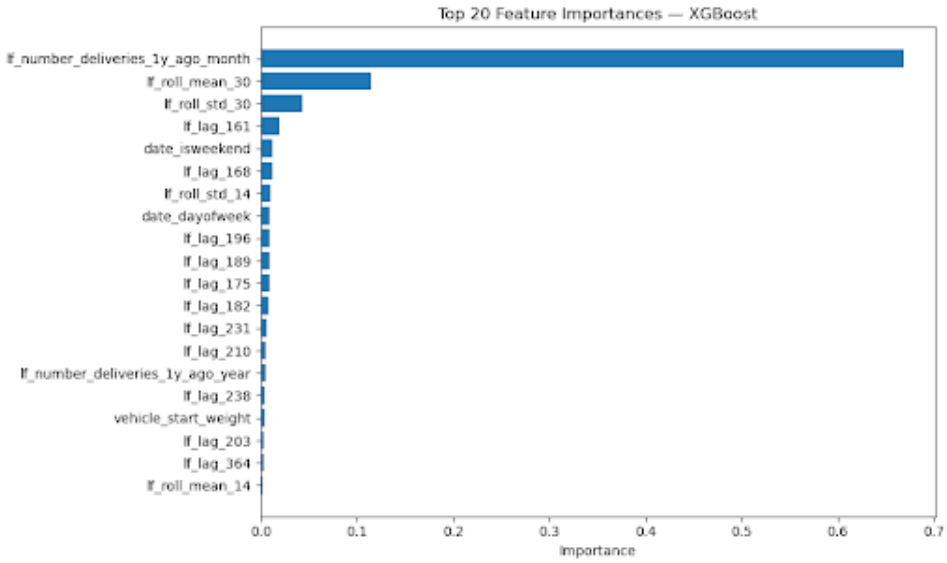

Figure 5: Feature Importance of the 20 Most Used Features

Observation: All these features result from our feature engineering efforts. The plot highlights many features related to the previous year, including lags, rolling means, rolling variances, and the number of deliveries. Notably, there are numerous weekly lags from the previous year. Interestingly, the static field "vehicle_start_weight" was utilized by the model despite lacking an obvious a priori connection to the target variable. Additionally, date-related features such as "date_isweekend" and "day_of_week" were also used, further emphasizing the strong weekly seasonality present in the data.

Conclusion: Date features related to the week prove to be highly valuable for the model. This underscores the importance of weekly patterns in predicting delivery behavior. The prominence of weekly features align.

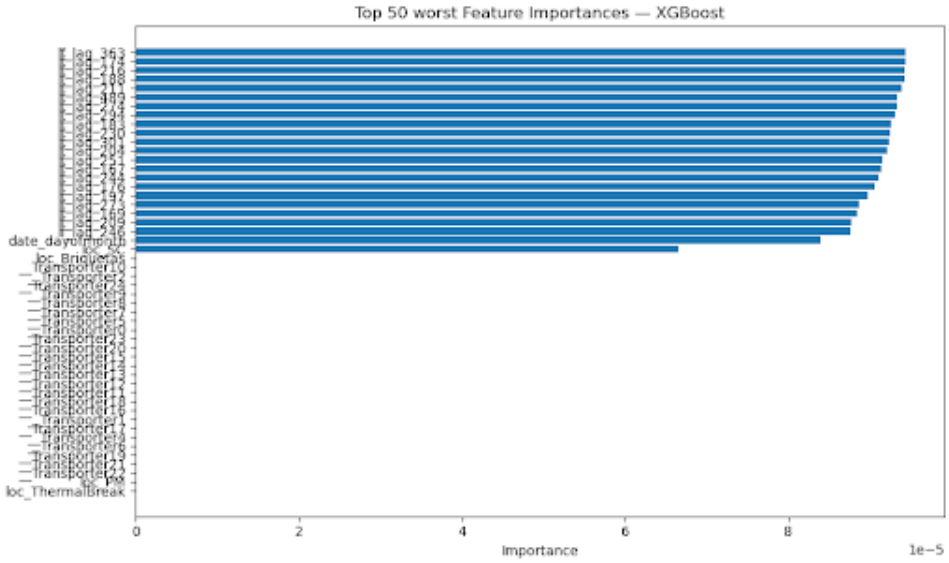

Figure 6: Feature Importance of the 50 Least Used Features

Observation: Many transporter dummy variables are not used at all, indicating that their values are either mostly or completely constant. The day of the month has little importance in our GXB model, and some lag features are irrelevant, as their importance values are below 1e-5. We also observed that the day of the month is not a relevant feature in our dataset. It is possible that we included too many lag features, rendering some of them irrelevant as others proved more effective.

Conclusion: Some transporter dummy variables are irrelevant because we initially removed certain rm_id values from the dataset. These rm_id values either lacked sufficient delivery data to estimate future deliveries or had not received deliveries for an extended period. Consequently, the remaining rm_id values in the dataset are never delivered by these transporters, resulting in constant and zero values for these dummy variables. The same conclusion applies to the features "loc_ThermalBreak" and "loc_Briquetas", which also exhibit constant values. To potentially improve the training time and performance of our model, we considered removing some of these less important features. However, after attempting to remove the 50 least important features, we consistently obtained worse results. Therefore, we decided to retain all features in the model.

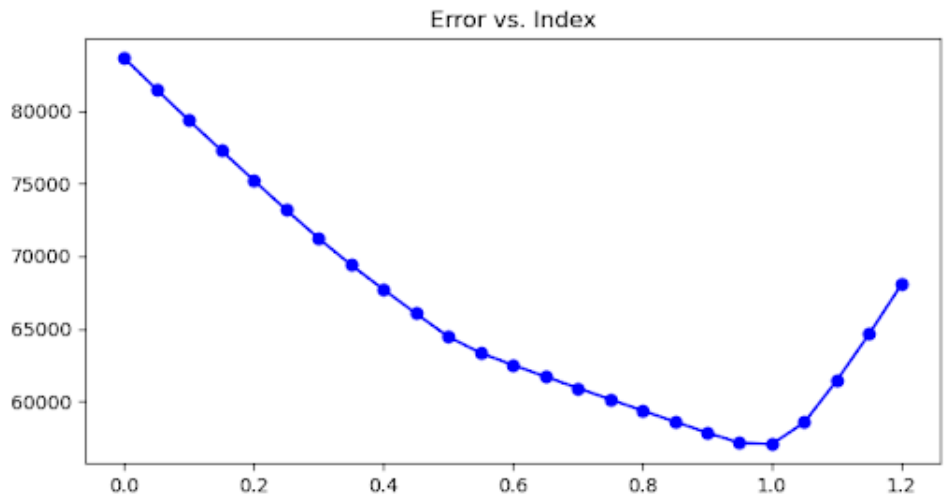

Figure 7: Quantile Error as a Function of Alpha

alpha is a coefficient that we apply to our prediction, so the new prediction is alpha*prediction. We use it to evaluate our model. If predictions equal reality, the value at alpha = 1 should be 0 and there are two straight lines at the right and at the left of 1 with different coefficients due to the asymmetric error function.

Observation: The best alpha value is 1, yielding an error of 57000. This is better than the baseline prediction (alpha = 0), which has an error of 85,000. The line to the right of alpha = 1 indicates that the model performs well for large deliveries. However, increasing the prediction leads to overestimation, which increases the error. The two lines to the left of alpha = 1 suggest that the model overestimates deliveries for certain rm_id values. The minimum error is 57000 remains far from the ideal value of 0.

Conclusion: Our model successfully predicts some deliveries with reasonable accuracy. However, the error remains significantly far from the ideal value of zero, with a minimum error of 57,000. We suspect that the model struggles to predict sporadic high deliveries. This issue likely arises because our loss function, combined with the nature of our data, penalizes errors more heavily when deliveries are infrequent. As an improvement, we propose developing a model that predicts the cumulative sum of deliveries. This approach would help mitigate the challenges posed by sporadic high deliveries. However, our attempts to build an effective cumulative-sum model have not yet succeeded. Further research and experimentation are needed to address this limitation.

6 Conclusion

This project aimed to forecast raw material deliveries for Hydro using historical data and purchase orders. After testing many approaches, we found that tree-based models, especially LightGBM and XGBoost, produced the most reliable results. They captured long-term seasonality and weekly delivery patterns, although both struggled with rare, very large deliveries.

I think we should have focused more on one model, such as XGBoost, instead of spending too much time trying to implement several complex models like the Temporal Fusion Transformer, which never worked properly. We were blocked many times trying to improve the score, but we realized too late that we should have invested more effort in feature engineering and understanding the data rather than in model diversity.

The project still shows that well-designed boosting models, combined with good features, can deliver accurate and conservative forecasts suited for industrial use. With more time, we could try to create an accurate model that forecasts directly the cumulative net-weight. We could also use a model of deep learning where we create all layers to have a specific model for our task.